Relatório 18 - Prática: Docker e Containers para Aplicações (III)

Matheus Beuren Carvalho

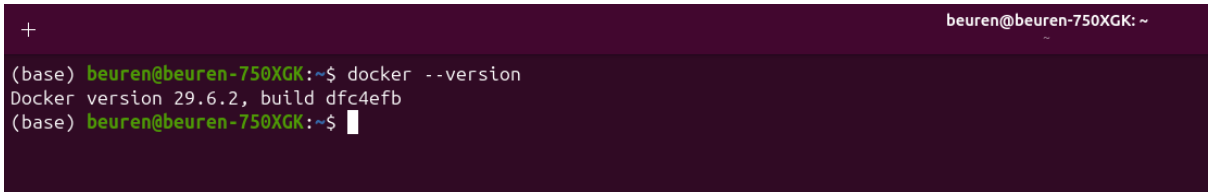
1. Introdução

O card desenvolvido se refere ao conteúdo de docker, uma das ferramentas mais importantes hoje no mundo de desenvolvedores. O intuito deste relatório é demonstrar os principais conceitos aprendidos e o caminho ao longo do curso de Docker.

2. Desenvolvimento

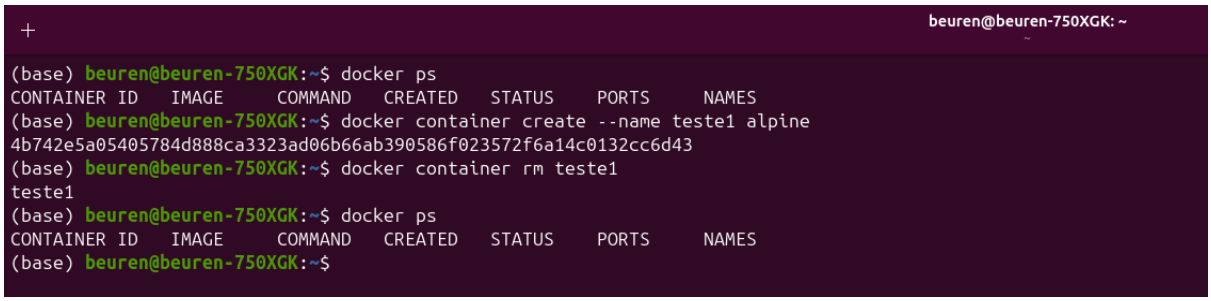
A primeira seção corresponde à instalação do docker, onde já tinha instalado na minha máquina linux, conforme mostra a figura 1.

Figura 1 - Instalação Docker



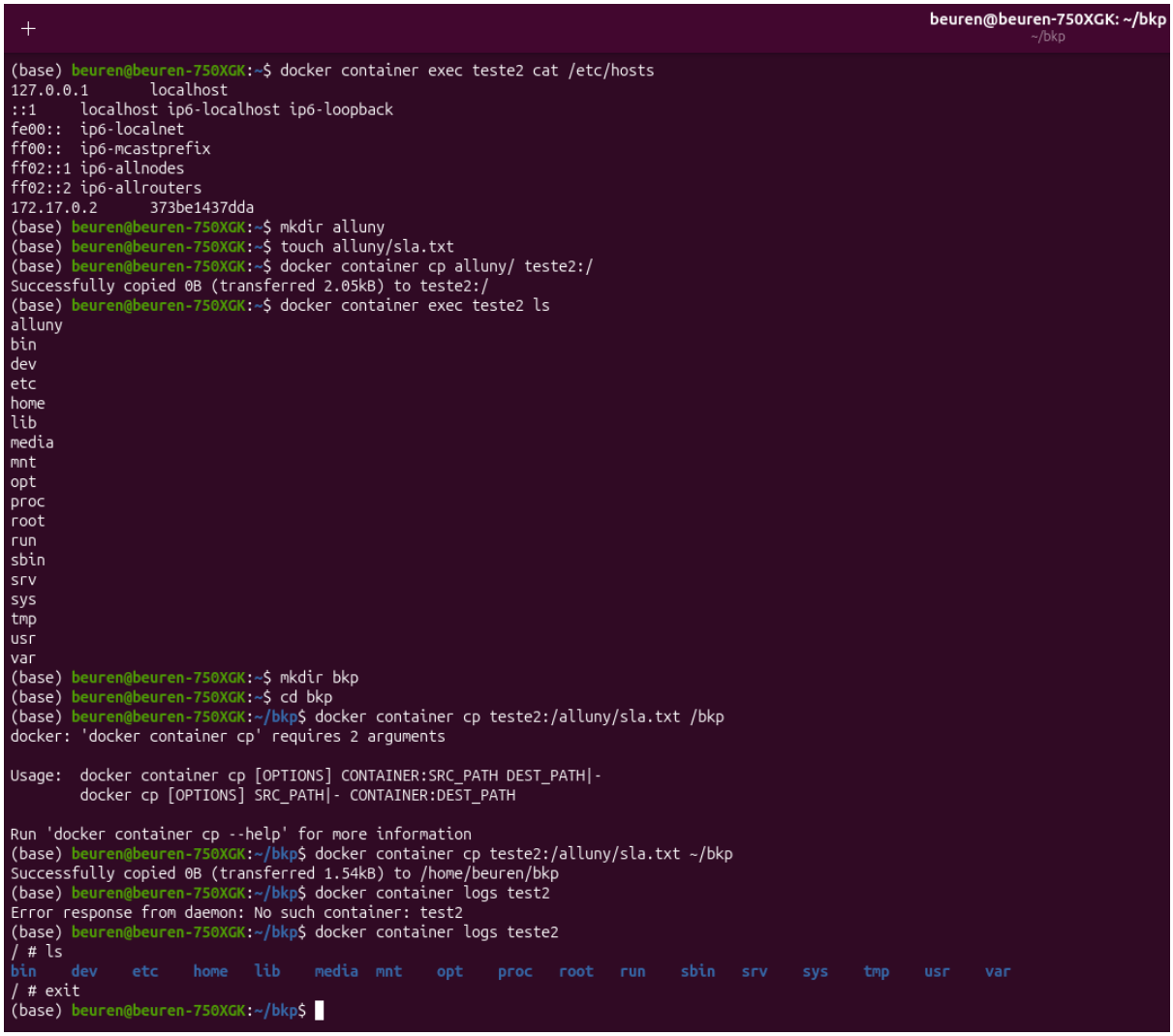
Em seguida foi introduzido ao Docker CLI(Command Line Interface), aos comandos antigos e novos. Na figura 2, é mostrado alguns comandos básicos com CLI.

Figura 2 - Docker CLI



Após a criação do container é possível entrar nele via terminal, e lá alterar arquivos e subir posteriormente uma aplicação. Ele é descartável, então após remover e criar novamente a imagem ou container o sistema é reiniciado e volta ao padrão caso tenha alguma alteração. A seguir é demonstrado como copiar arquivo local para a o container e trazer do container para o local.

Figura 3 - Manipulação do container



Os volumes em Docker servem para armazenar dados mesmo após o desligamento do container, e também um meio de colocar dados do host para o container. Seu principal uso é para que, mesmo após o desligamento os dados se mantenham do container. Diversos comandos, como criar, remover e associar um volume a um container em sua criação foram apresentados e realizados na aula. Na figura 4 é visto como anexar um volume já criado a um container.

Figura 4 - Anexando volume a um container

```
(base) beuren@beuren-750XGK:~/Documentos/LAMIA-Bootcamp/card-18/pratice$ docker volume create meu_vol
meu_vol
(base) beuren@beuren-750XGK:~/Documentos/LAMIA-Bootcamp/card-18/pratice$ docker run -v meu-vol -p 8080:80 test-html:latest
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/08/17 17:22:12 [notice] 1#1: using the "epoll" event method
2026/08/17 17:22:12 [notice] 1#1: nginx/1.31.3
2026/08/17 17:22:12 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/08/17 17:22:12 [notice] 1#1: OS: Linux 7.0.0-29-generic
2026/08/17 17:22:12 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1024:524288
2026/08/17 17:22:12 [notice] 1#1: start worker processes
2026/08/17 17:22:12 [notice] 1#1: start worker process 29
2026/08/17 17:22:12 [notice] 1#1: start worker process 30
2026/08/17 17:22:12 [notice] 1#1: start worker process 31
2026/08/17 17:22:12 [notice] 1#1: start worker process 32
2026/08/17 17:22:12 [notice] 1#1: start worker process 33
2026/08/17 17:22:12 [notice] 1#1: start worker process 34
2026/08/17 17:22:12 [notice] 1#1: start worker process 35
2026/08/17 17:22:12 [notice] 1#1: start worker process 36
2026/08/17 17:22:12 [notice] 1#1: start worker process 37
2026/08/17 17:22:12 [notice] 1#1: start worker process 38
2026/08/17 17:22:12 [notice] 1#1: start worker process 39
2026/08/17 17:22:12 [notice] 1#1: start worker process 40
```

Por padrão o container cria um rede virtual onde cada contêiner recebe um ip nessa rede, existindo somente na máquina local do host. Ao configurar um ip host como ‘8080’ e ‘80’ no container, ao tentar acessar na máquina do host a porta ‘8080’ será direcionado a porta ‘80’ que será atribuída a algum serviço, como no exemplo a seguir, o nginx.

Figura 5 - Mapeamento de portas na criação do container

```
                                                                    beuren@beuren-750XGK: ~
(base) beuren@beuren-750XGK:~$ docker container run -d -p 80:80 --name nginx nginx
4210ad02201c0d3dde2796f7e948b2d45861ab5964113384620564def6639618
(base) beuren@beuren-750XGK:~$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS          PORTS                                   NAMES
4210ad02201c   nginx     "/docker-entrypoint.…"   2 seconds ago    Up 2 seconds    0.0.0.0:80->80/tcp, [::]:80->80/tcp   nginx
373be1437dda   alpine    "sh"                     59 minutes ago   Up 30 minutes                                           teste2
(base) beuren@beuren-750XGK:~$ docker container inspect nginx
```

Dado um contêiner em funcionamento que tenha algum serviço ou modificações, é possível criar uma imagem a partir deste container e poder mandar outros usuários para rodar seu container da mesma forma com docker engine. Também é possível exportar e importar imagens a partir de arquivos ‘.tar’ facilitando a utilização de containers prontos.

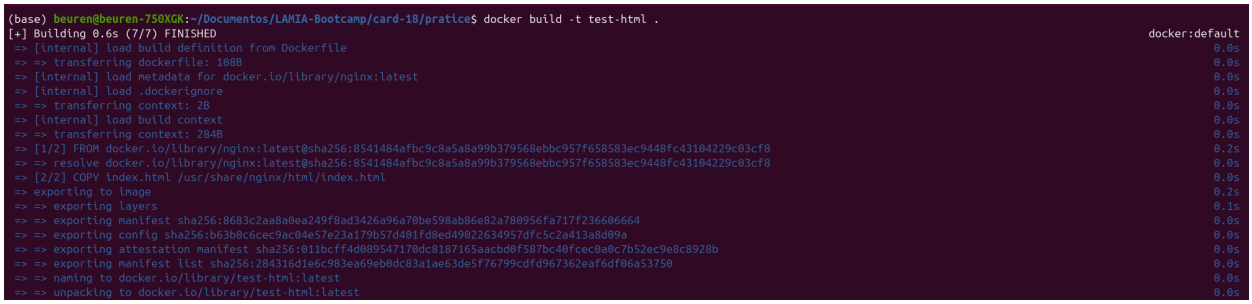
O comando docker exec -it faz com que possamos entrar dentro de um container para executar comandos e manipulá-lo, como pode ser visto na figura 6.

```
matheus@LAPTOP-I0U9BL8N:~/LAMIA/Bootcamp-Aprendizado-de-Maquina-LAMIA/card-18/pratice$ docker run -it alpine sh
/ # ls
bin    dev    etc    home   lib    media  mnt    opt    proc   root   run    sbin   srv    sys    tmp    usr    var
/ #
```

No exemplo mostrado ao entrar em um container que não possui o bash por padrão, mas com sh. Para entrar com shell é necessário somente trocar o sh por bash

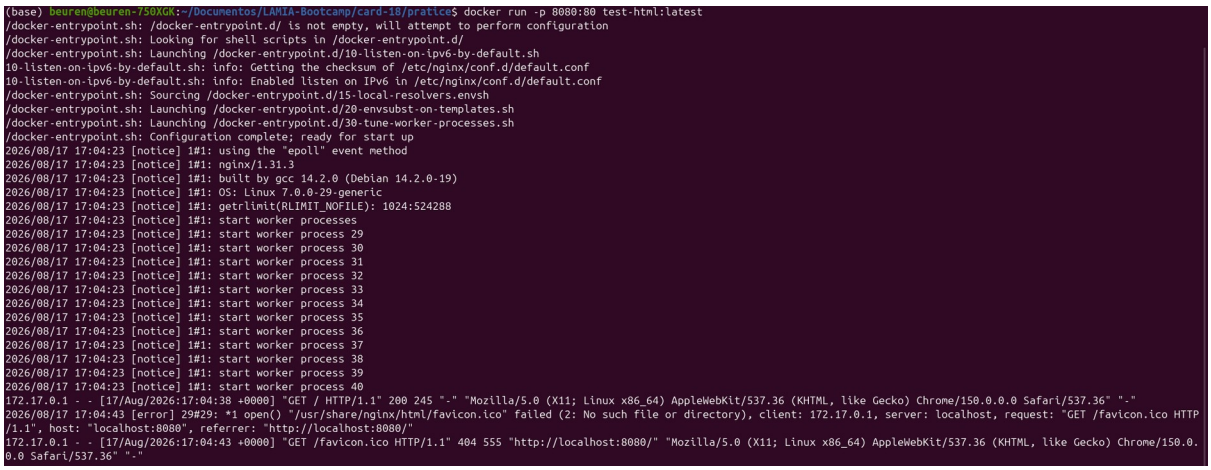
O dockerfile é um arquivo que em forma de código é possível escrever as especificações de um container como sua imagem base, dependências e comandos para se executar após o container subir. Para a prática de dockerfile utilizei um código simples em html e criei uma imagem via dockerfile, assim também executei o container com a imagem construída e pode ser vista na web, a sequência de figuras 5 e 6, mostram os comandos utilizados para fazer a prática

Figura 7 - Construindo a imagem a partir do dockerfile



O comando exige que tenha um dockerfile e no comando em si, o nome da imagem que será criada e o “.” significa que é para utilizar todos os arquivos da pasta atual(Dockerfile e index.html).

Figura 8 - Executando um container com a imagem criada



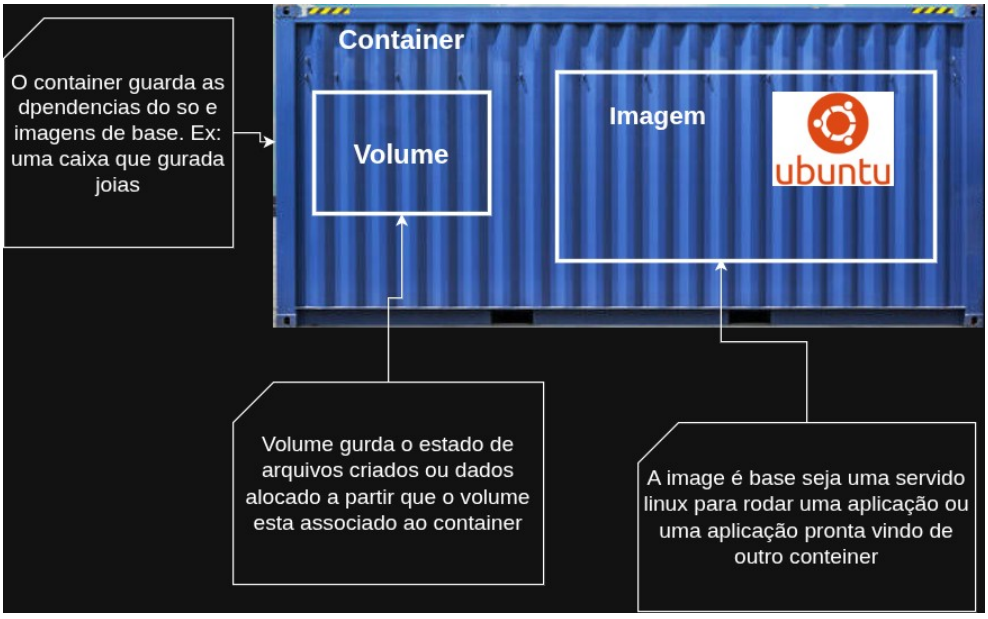
A página em html pode ser vista na rede local criada pelo docker na porta 8080.

Alguns comandos como System mostram o desempenho, monitoramento e como vai o sistema em si da engine rodando, um exemplo é o atributo df que mostra qual o consumo do docker na máquina.

Figura 9 - System df

```
matheus@LAPTOP-I0U9BL8N:~/LAMIA/Bootcamp-Aprendizado-de-Maquina-LAMIA/card-18/pratice$ docker system df
TYPE            TOTAL     ACTIVE    SIZE      RECLAIMABLE
Images          27        10        24.58GB   5.89GB (23%)
Containers      14        0         2.205GB   2.205GB (100%)
Local Volumes   57        7         9.606GB   4.177GB (43%)
Build Cache     98        0         9.143GB   3.333GB
```

Figura 10 - Arquitetura básica do Docker



3. Conclusão

O docker as questões de dependências de aplicações em diferentes máquinas e facilita implementação de aplicações com maior agilidade. Resolve o principal problema que em algumas situações a aplicação roda na máquina linux mas não no MacOS, o container guarda as dependências necessárias para a aplicação rodar e seu estado em que a imagem foi criada. Isso facilita o trabalho em equipe e principalmente o deploy de uma aplicação em uma nuvem.